Screenshot Testing Strategy: BILD · Sport BILD · WELT News · WELT Edition

Goal: catch unintended visual changes on every pull request, on iOS and Android, by rendering real production UI with mocked data that is identical on every run.

In one line: native in-process snapshot tools on each platform, fed by **one shared set of JSON fixtures**, gated in CI on every PR. A nightly pass on real simulators and emulators covers what in-process rendering cannot see.

The demo in this repo implements the iOS side end to end (app, tests, CI); §10 shows how Android does the same with the same fixtures. Evidence from building and running it: docs/EVIDENCE.md.

1. Tooling

	iOS	Android
PR gate (Layer 1)	swift-snapshot-testing 1.19.7 (pinned)	Roborazzi 1.77 on Robolectric 4.17 (pinned)
Why	In-process and fast. Same authors as TCA, so mocking reuses `withDependencies`	JVM, no emulator, works with Hilt. The same run drives interaction tests
Also in the gate	Logic/contract tests; XCUITest interaction tests and `performAccessibilityAudit()`	Logic/contract tests; Robolectric interaction tests and Google ATF checks
Nightly (Layer 2)	Maestro/XCUITest on a simulator, app started with `-fixture <name>`	Maestro on an emulator, `--es fixture <name>`

Rejected for the gate: Paparazzi (2.0 alpha, no Hilt or interactions; it is the fallback, because the test code is a thin layer over portable fixtures), Compose Preview Screenshot Testing (alpha), and Appium/Maestro with a paid visual service (minutes per screen, flakier, edge states hard to reach: that is Layer 2). Hosted review (e.g. Sentry Snapshots) is worth a later trial.

2. Mocked, deterministic data

- **News changes every minute; goldens must not.** A golden belongs to a *fixture*, never to live content, so it cannot go stale: fixture times are relative to a pinned clock ("vor 9 Min." forever). Changing data is tested by other means:
  - **contract tests** of the decoders, nightly against anonymised real responses, catch format drift
  - **fixture refreshes** from real responses happen in their own reviewed PR, which shows exactly which goldens it re-baselines
  - **robustness tests without goldens:** a seeded fuzz test corrupts one feed item 300 ways, and every other item must still show
- **One fixture catalog, decoded by each platform's production decoder:** typical, long German headlines, empty, edge cases and hostile content (HTML entities, stray line breaks, soft hyphens, emoji, a ticker with no entries, a 1:02:03 video, a timestamp in the future). The edge-case and hostile-content fixtures are **contracts**, asserted as exact literals that every platform's decoder must produce: a malformed item is dropped, never the whole feed; missing flags get safe defaults; duplicate ids keep the first entry. Without this, two decoders silently diverged in an earlier two-platform version: one missing field blanked one feed but made a PLUS article free in the other.
- **Every source of variation is pinned:** clock, `de_DE` locale, `Europe/Berlin` time zone, seeded randomness (a portable RNG), animations off, local images, fixed geometry, theme and text size per variant, and placeholders for ads and WebViews.
- **Seams.** UI and view models depend on small client values (`FeedClient`, `AuthClient`, an image provider), never on the HTTP stack. In the existing apps (stack *assumed from public job ads*):

Seam	iOS	Android
Data	TCA `@DependencyClient` → `withDependencies { $0.feedClient = .fixture(…) }`	Hilt `@TestInstallIn` replaces the network module
Images	Kingfisher/Nuke pipeline with a synchronous fixture loader	Coil `FakeImageLoaderEngine`
Legacy code (UIKit, singleton API client)	`URLProtocol` stub serving `/fixtures`: no refactor needed to start	OkHttp `MockWebServer` serving the same files

3. Golden images: storage, review, guards

- **Storage:** goldens live next to the tests (Git LFS beyond about 100 MB); test and golden names are identical across platforms.
- **Only CI records goldens.** A developer adds the `record-snapshots` label; CI re-records on the pinned image, **proves the new goldens** (lint plus a full verify) and pushes them as a bot commit; design/QA code owners approve the golden diff, and stale approvals are dismissed. This ran for real when the iOS simulator runtime had to be re-pinned for CI.
- **Review:** one sticky PR comment, updated in place, with an HTML diff gallery (golden, this PR, changed pixels). The comment classifies each change before anyone opens an image: **layout** (size changed, something moved), **visible**, or **subtle** (colour or anti-aliasing only). This is triage, never approval. `make gallery` puts every approved screenshot on one page for design review (with two platforms: side by side, because each gate only compares a platform with itself).
- **Guards against a green test that protects nothing:** golden lint (blank goldens; with two platforms, name parity), a capture guard against undrawn frames, fixture-precondition tests ("the premium teaser is premium"), and recording from an empty folder so orphaned goldens show up as deletions. Every new guard is first shown to fail on a broken input.

4. What is covered, and by which layer

- **Matrix (curated, not every combination):** every component in light + dark; text-heavy and interactive screens also at the largest supported text (iOS AX3, Android font scale 2.0 with non-linear scaling); error, selected and interactive states also in dark; the feed also on the smallest phone, a tablet and at **full length**; paywall and PLUS badge always. Demo: 89 goldens.
- **Accessibility in the gate:** a WCAG contrast test over every colour-token pair, plus automated audits on every screen (contrast, touch targets, labels, traits). The audit found that hidden tabs were still reachable by VoiceOver, invisible in every screenshot. Accepted audit exceptions are listed in code with the evidence for each.
- **Layer 2 (nightly):** what in-process rendering cannot see: system chrome (iOS 26's Liquid Glass tab bar renders blank in-process), real fonts and devices, and hostile settings (en_US 12-hour time, bold text, increased contrast). It found an "11:05 PM Uhr" bug no snapshot could.

5. CI/CD and the Scrum process

- **When it runs:** every PR on its merge result (golden lint and iOS on the `xcode-27` runner; Android on Linux in parallel); the merge queue, so two green PRs cannot break main together; main after merge; nightly on a fresh runner image to catch toolchain drift (switched off in the demo repo to save CI minutes).
- **Measured on GitHub Actions:** iOS 12–19 min (snapshots, UI tests and audits). Cost control: run iOS only when iOS or shared files change, build once for snapshots and UI tests, move the slowest audits to nightly. Target: under 10 minutes per PR.
- **Rollout of the gate:** report-only first, then a required check. **Definition of Done:** a new UI state ships with a fixture and a snapshot; refinement asks "which states does this add?".
- **Platform upgrades** (Xcode, simulator runtime, SDK, Robolectric, the snapshot library): one PR bumps the version *and* re-baselines, reviewed as such.

6. Stability

1. **Pin the environment:** Xcode, simulator device *and runtime*, runner image, and all library and test dependencies. The first CI run failed because the pinned runtime was not on the CI image; the pin is now a runtime that exists on both, fetched exactly or failing loudly.
2. **Pin the inputs** (§2). A `Date()`, network image, animation or unseeded random value in a view is a test-

design bug.

3. **Tolerance:** iOS uses `perceptualPrecision 0.98`, which absorbs GPU anti-aliasing but fails on a 2pt shift. Android (§10) allows no changed pixel beyond Roborazzi's per-pixel colour tolerance. Tolerance is never loosened globally; a per-test exception needs a comment and review. **No "small diff = auto-pass":** a 2pt shift and a dropped PLUS badge are both small; intent is a product decision, not a pixel count.
4. **Local equals CI.** CI is the source of truth and records goldens; a Mac verifies the CI-recorded goldens, so every CI failure reproduces locally.
5. **No automatic retries.** A flaky snapshot is quarantined within 24 hours with an owner; target flake rate under 1%. Measured: 10 consecutive full iOS runs, all green.

7. Integration into the existing apps

- **Fixtures** live in one shared `news-fixtures` package (SPM resources plus a Gradle artifact), versioned with the API contract and seeded from anonymised real responses.
- **First two weeks, without blocking feature work:** snapshot design-system components that take plain models (no seam needed); add a `URLProtocol`/`MockWebServer` stub as a stopgap; add a client seam at one feature boundary; run the CI job report-only.
- **Never change product UI for a test tool.** The demo's tab bar is custom so every pixel is testable; in production, keep the system tab bar, snapshot the tab *content*, and cover the chrome in Layer 2.
- **A test-support module per platform** (deterministic wrapper, variants, capture guard, diff report) makes a new snapshot test about 5 lines.

8. Scaling, ownership, enablement

- **Four apps, one design system:** component goldens are shared; app modules (WELT PLUS, Sport BILD tickers) own their screens and fixtures.
- **Ownership:** developers write and maintain the tests. QA owns the strategy, the fixture catalog and the matrix rules. Design approves goldens. AI may triage diffs; a human approves.
- **Enablement:** an "add a snapshot in 5 lines" guide, pairing with each squad, a diff-review session for designers, office hours in month one.
- **Metrics:** escaped visual bugs per release, PR gate time, flake rate, share of UI states with fixtures, accessibility findings per release.

9. Rollout (about 4 sprints)

Sprint	Step	Exit criterion
1	One app (e.g. WELT News, iOS + Android): support module, 5 design-system components, CI **report-only**	Green for 5 days, no flakes
2	Shared fixtures, feed and paywall states, diff gallery and sticky comment, CODEOWNERS; pairing with each squad	Every squad merged at least one snapshot test
3	Gate **required** and in the Definition of Done; CI-only recording; Layer 2 nightly	Under 10 min, flake rate under 1%
4+	Other three apps; accessibility snapshots; trial a hosted review tool	Visual bugs caught by the gate, not in release testing

10. Android: the same approach

- **Same fixtures, same contracts, same names.** Android reads the same `fixtures/` JSON with its production decoder, asserts the same contract literals, and uses the same test and golden names, so one regression fails in the same places on both platforms; golden lint then also checks iOS/Android name parity.
- **Tool:** Roborazzi on Robolectric, rendering Compose on the JVM (no emulator, about 20 s for the full suite in an earlier Android version of this demo). Seams via Hilt (`@TestInstallIn`) and Coil's fake image engine (§2). The same run drives interaction tests and Google ATF accessibility checks.
- **CI:** a Linux job in parallel with iOS; the diff report needs CoreGraphics, so it runs on a short macOS job (or is ported).
- **Lessons from building it:** pin the Robolectric SDK (a newer one rendered blank captures that still passed), pin transitive test dependencies, prove accessibility checks can fail (one official hook silently checked nothing), and keep tab state outside the composables.